Prob 1. (20 points, 4 each) Determine what mode (push-pull, open-drain, pull-up, and pull-down) you will use for the following situations:

1. Input, seeing HiZ as logic 1.
2. Input, seeing HiZ as logic 0.
3. Output, drawing current from the load connected between the output pin and a voltage with high voltage.
4. Output, turning on the LED when the output is logic 0 on the GPIO pin connected to the LED.
5. Output, turning on the LED when the output is logic 1 on the GPIO pin connected to the LED.

Solution:

1. Input, seeing HiZ as logic 1. **Pull-up**
2. Input, seeing HiZ as logic 0. **Pull-down**
3. Output, drawing current from the load connected between the output pin and a voltage with high voltage. **Open-drain**. Push-pull can work, but open-drain is a better choice.
4. Output, turning on the LED when the output is logic 0 on the GPIO pin connected to the LED. **Open-drain**. Push-pull can work, but open-drain is a better choice.
5. Output, turning on the LED when the output is logic 1 on the GPIO pin connected to the LED. **Push-pull**

> -:-

Remarks about the solution:

- We need to make sure that pull-up and pull-down are used only for the case for input. Hence, Parts 1 and 2 will use pull-up or pull-down. As for which to use, it depends on the situation. If we want to force HiZ, which is a floating input, to be logic 1, we need to *tie it up* to logic 1, the voltage source. In the drawing of circuit diagram, the voltage source is above the ground, which is logic 0, so we need to use pull-up. On the other hand, if we want to force HiZ to be logic 1, we need to *tie it down* to logic 0, the ground, so we need to use pull-down.
- When we deal with output, we need to use push-pull or open-drain. The comparison of those two is provided in Section 6.2.7.1 of Lctr 6. Here are the points to stress:
  - If you need to *push* a current to the load, which means the current flows from the MCU to an external device, we have to use the push-pull mode.
  - Both push-pull and open-drain modes can be used to draw a current from the load to the MCU. Yet, open-drain is mostly used for this case.
  - A deep understand of the topic requires a good understanding of how transistors work. We will not go to all the details of the topic. Just remember if you can use open-drain, then use it. If you have to use push-pull, then use it.

> -:-

Prob 2. (20 points, 5 each) In Section 6.5 in Lctr 6' notes, we have discussed that we can arrange a bank of GPIO registers of a certain GPIO port in a data structure. We have also discussed that we can define a pointer to point to the register bank of a certain GPIO part. Determine the following:

1. The (start) address of GPIOC if the GPIO ports are on AHB2.

2. The address of the IDR of GPIOC if the GPIO ports are on AHB2.
3. The (start) address of GPIOC if the GPIO ports are on AHB1 (without other changes).
4. The address of the ODR of GPIOC if the GPIO ports are on AHB1.

Solution:

1. When the GPIO ports are on AHB2, the address of GPIOC is `0x4800_0800`.
2. When the GPIO ports are on AHB2, the address of IDR of GPIOC is `0x4800_0810`.
3. When the GPIO ports are on AHB1, the address of GPIOC is `0x4002_0800`.
4. When the GPIO ports are on AHB1, the address of ODR of GPIOC is `0x4002_0814`.

> -:-

Remarks about the solution:

- We know that the MCUs we are dealing with use 32-bit address. Hence, the values of the pointers are all 32-bit numbers. GPIOC is defined as a pointer to GPIO port C with the value of the address being `GPIOC_BASE`, which is `AHB2PERIPH_BASE + 0x0800UL`. Checking back, we see that `AHB2PERIPH_BASE` is defined as `PERIPH_BASE + 0x08000000UL`. So, adding the value, we have `GPIOC_BASE` equals `0x4800_0800`.
- We can see from the definition of the data structure of GPIO that IDR has an offset of 0x10. Hence, the address of IDR for GPIOC is `0x4800_0810`.

> -:-

Prob 3. (10 points) Given `uint8_t A = 0b1010_1011;`. What is the value of `uint8_t B = (A >> 2) << 2;` in binary?

Solution:

```
B = 0b1010_1000
```

> -:-

Remarks about the solution:

- When we shift `A` to the right by two bits, we shift out the least significant two bits and shift in two zeros from left so that `A >> 2` is `0b0010_1010`.
- When we shift `0b0010_1010` to left by two bits, we shift in two zeros from the right to obtain the result we have.

> -:-

Prob 4. (10 points) What is the value of `B` in binary after running the code below?

```
uint8_t A = 0b1010_1011;
uint8_t B = (A << 2) >> 2;
```

Solution:

```
B = 0b0010_1011
```

> -:-

Remarks about the solution:

- When we shift `B` to the left by two bits, we shift out the most significant two bits and shift in two zeros from the right so that `B << 2` is `0b1010_1100`.
- When we shift `0b1010_1100` to right by two bits, we shift in two zeros from the left as the number is unsigned to obtain the result we have.

> -:-

Prob 5. (10 pts) Assume we want to control the ODR of a GPIO port. If we need to toggle Pin5 and Pin7, what will be the mask we need to use in hexadecimal?

Solution:

```
mask = 0x00A0
```

> -:-

Remarks about the solution:

- The mask is always related to the bits we want to operate on. Here we operate on Pin5 and Pin7, which are on Bit 5 and Bit 7, respectively according to the structure of the ODR, shown on Figure 10 in Lctr 7.
- The mask for Bit 5 has 1 on Bit 5 and zero for other bits, and hence it is `0b0000_0000_0010_0000`.
- Similarly, the mask for Bit 7 is `0b0000_0000_1000_0000`.
- If we want to toggle these two pins, we can perform the action in one line of code, as shown in `mp_toggle_bits` in Section 7.3.8 of Lctr 7. For this purpose, we can combine, using bitwise OR, the above two masks together to form `0b0000_0000_1010_0000` which can be written as `0x00A0`.

> -:-

Prob 6. (10 points) Given `uint8_t A = 0b1010_1011;`. What are the values of `uint8_t B = A | (4U);` and `uint8_t C = A & ~(4U);` in binary, respectively?

Solution:

```
B = 0b1010_1111
C = 0b1010_1011
```

> -:-

Remarks about the solution:

- The least significant 8 bits of `4U` is `0b_0000_0100`. Hence, its bitwise OR operation with `0b1010_1011` is `0b1010_1111`. Now, we effectively set bit 2 of `A`.
- The least significant 8 bits of `~(4U)` is `0b_1111_1011`, and its bitwise AND operation with `0b1010_1011` is still `0b1010_1011` as the intention of this operation is to clear bit 2, which was 0 at the first place.

> -:-

Prob 7. (10 points) Consider the control of OTYPER for GPIOB. Answer the following questions:

1. (5 points) If the value of OTYPER of GPIOB is `0x0000_1234`. What is the value after the statement `GPIOB->OTYPER |= (1U << 3)`. (Give your solution in hexadecimal.)
2. (5 points) If we want to change the output type of Pin 4 to push-pull, write a ONE-line C statement like the one given above to perform this.

Solution:

Part 1.

```
0x0000_123C
```

Part 2.

```
GPIOB->OTYPER &= ~(1U << 4)
```

> -:-

Remarks about the solution:

- `GPIOB->OTYPER |= (1U << 3)` will set bit 3 on the register to 1 without affecting others. The given value of the least significant 4 bits of the register is `0b0100`. By setting its bit 3, it is changed to `0b1100`, which is `0xC`. Hence, we have the given result above.
- According to Section 7.4.2 of Lctr 7, if we want to configure the output type of Pin 4 to push-pull, we need to clear bit 4 of OTYPER of GPIOB. Hence, we use the above one-line code.

> -:-

Prob 8. (30 points total, 6 points each) Assume `A` is a `uint16_t` variable, written as 0x$H_3H_2H_1H_0$, intended to express an arbitrary number in hexadecimal format. Here H_k is the kth hexadecimal digit written in the normal way where the right bit is the least significant bit. Write the C code to perform the following operations to change the digit (4 bits) of interest without affecting the other digits (bits) or shift the number following the example given for (a):

(a). A = 0x$H_3H_2H_1H_0$ => A = 0x$H_3H_20H_0$ (for example A = 0x1234 => A = 0x1204)

(b). A = 0x$H_3H_2H_1H_0$ => A = 0x$H_3H_2FH_0$

(c). A = 0x$H_3H_2H_1H_0$ => A = 0x$H_3H_26H_0$

(d). A = 0x$H_3H_2H_1H_0$ => A = 0x$H_3H_29H_0$

(e). A = 0x$H_3H_2H_1H_0$ => A = 0x$H_3H_2\bar{H}_1H_0$ where 0x$\bar{H}_1$ = 15− 0xH_1 (hint: recall the XOR operation)

(f). A = 0x$H_3H_2H_1H_0$ => A = 0x0$H_3H_2H_1$

Illustration for (a):

```
    A &= 0xFF0F;
// or better:
    A &= ~(15u << 4);
```

Solution:

```
(b). A |= 0x00F0;
(c). A &= 0xFF0F;
     A |= 0x0060;
(d). A &= 0xFF0F;
     A |= 0x0090;
(e). A ^= 0x00F0;
(f). A >>= 4;

Or the preferred soln:
(b). A |= 15u << 4;
(c). A &= ~(15u << 4);
     A |= 6u << 4;
(d). A &= ~(15u << 4);
     A |= 9u << 4;
(e). A ^= 15u << 4;
(f). A >>= 4;
```

> -:-

Remarks about the solution:

- The difficulty of this problem is to decipher the notation. The key is to understand, according to the given text, that H_k is the kth hexadecimal digit written in the normal way. So, H_2 it digit 2 of the hexadecimal number. Its value can be anything from 0 to `0xF`.
- In part (a), we give an example to convert A = 0x$H_3H_2H_1H_0$ to A = 0x$H_3H_20H_0$. This means that we only zero FOUR bits, bits 4 to 7, but do not change other digits. Hence, we provided the code for the corresponding bitwise operation:

  ```
  A &= 0xFF0F;
  // or better:
  A &= ~(15u << 4);
  ```

- In part (b), A = 0x$H_3H_2H_1H_0$ => A = 0x$H_3H_2FH_0$, we want to set FOUR bits, bits 4 to 7, but do not change other digits. Hence, we need to use the bitwise OR operation with a mask for four bits. To have a mask for `N` bits, we can use the expression `(1U << N)- 1`. For `N` equals to 4, this value is `(1U << 4)- 1` = `16- 1= 15`. As we need to work on bits 4 to 7, we need to shift the able mask to the left by 4 bits, to obtain `15U << 4`. The rest is easy to understand.
- In part (c), A = 0x$H_3H_2H_1H_0$ => A = 0x$H_3H_26H_0$, we need to assign 6 to bits 4 to 7. This is similar to what we do in the `mp_assign_bit` function in Section 7.3.9 in Lctr 7 as well as the code in Section 7.4.1.
- Part (d) is just a repetition of Part (c) with a different number.
- For Part (e), the difficulty is to recognize that the needed operation is just the bit-toggle operation. Use

the values of 6 and 9 in Parts (c) and (d) to verify this. Once this verified, we just need to follow the code in the `mp_toggle_bits` function in Section 7.3.8.

- Part (f), A = 0x$H_3H_2H_1H_0$ => A = 0x0$H_3H_2H_1$, is just a right-shift operation. As each digit has 4 bits, we need to shift to the right by 4 bites.

> -:-

Prob 9. (15 pts) Use an LLM to learn the following aspects of wire-AND: (1) what are open-drain and open-collector circuits for digital output? (2) why open-drain or open-collector circuits can be wire-ANDed? (3) what if we connect the circuits of push-pull output as if they are open-drain? what will happen for this connection? Submission:

- (For online submission) Please copy the responses of the LLM to your questions.
- Summarize your understanding after fact-checking to short answers for the above questions.

Solution:

(1) What are open-drain and open-collector circuits?

- **Open-collector (BJT logic, TTL)** and **open-drain (MOS logic, CMOS)** outputs are very similar concepts:
  - The output stage **does not actively drive the line high**.
  - Instead, it can **pull the line low** (sink current) or **leave it floating** (high-impedance, "open").
  - To get a valid logic "1," you need an **external pull-up resistor** to V_{CC}.
- Both behave the same way logically, with a bit difference:
  - **Open-collector** → uses a BJT transistor with collector left "open."
  - **Open-drain** → uses a MOSFET with drain left "open."

(2) Why can open-drain/open-collector circuits be wire-ANDed?

- If you tie several open-drain/collector outputs together on the same line with a **single pull-up resistor**:
  - Any device can pull the line **low** → logic "0" dominates.
  - If all devices release (go high-impedance), the pull-up resistor brings the line **high** → logic "1".
- This matches the behavior of a **wired-AND** (in positive logic):
  - 0 AND anything → 0
  - Only when *all outputs are "1"* (all released) does the line go high.

(3) What if we connect push-pull outputs as if they are open-drain?

- A **push-pull output** can drive both **high** and **low** actively.
- If you connect two push-pull outputs together:
  - If both drive the same value → OK.

  - If one drives **high** and the other drives **low** $\rightarrow$ you get a **direct short** between V_{CC} and ground. This causes **high current**, possible damage, or at least logic errors.

In short:

- **Safe with open-drain/collector** because only one side ever drives low, and high is "released."
- **Dangerous with push-pull** because both high and low are actively driven, leading to bus contention.

Manual summary:

1. Open-drain (MOSFET) and open-collector (BJT) are outputs that can only pull low or float, requiring a pull-up resistor, which works in the same way as the pull-up resistor in the input path of the GPIO port.
2. They can be wire-ANDed since any device pulling low forces the output to be low, otherwise the pull-up makes it high.
3. Push-pull outputs cannot be wire-ANDed safely; contention occurs if different devices drive opposite levels. Contention can damage the device.

> -:-